7. Neural Networks

An artificial neural network is a network of simple processing units called **neurons** (or nodes) wired together in layers, loosely inspired by the human brain.

1. How Logistic Regression Connects to Neural Networks

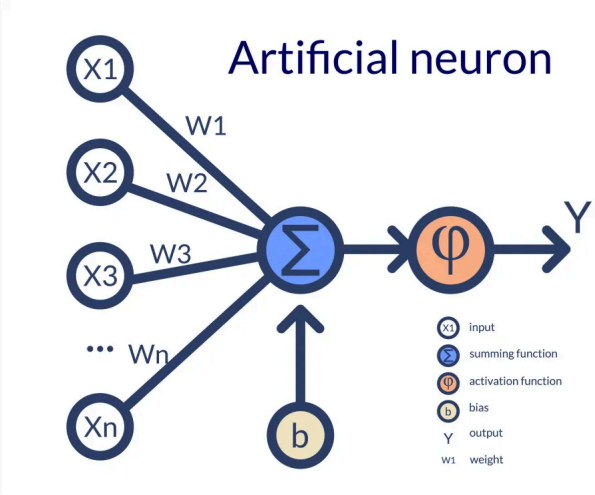
Logistic regression is simply an artificial neural network with **a single neuron**.

$$\text{Input Features } (x_1, x_2, \dots) \longrightarrow \Big[\text{Weighted Sum } + \text{ Bias}\Big] \longrightarrow \Big[\text{Sigmoid Function}\Big] \longrightarrow \text{Output Probability } (\hat{y})$$

- **A Single Neuron:** Takes multiple inputs, assigns a weight to each, adds a bias, and runs the result through a Sigmoid curve to output a probability between 0 and 1. That is the exact mathematical definition of logistic regression.
- **A Neural Network:** Takes that single building block and duplicates it hundreds or millions of times. By arranging neurons into an **input layer**, one or more **hidden layers**, and an **output layer**, the network can solve problems with complex curves and non-linear patterns that a single logistic regression unit cannot handle.

2. Inside a Neuron: Aggregation and Activation

Every individual neuron in a network processes information in two distinct, sequential steps:



Step 1: Aggregation (The Weighted Sum)

The neuron gathers all incoming signals, scales each one by its importance, and adds a baseline offset:

$$z = \sum_{i=1}^{n} (w_i \cdot x_i) + b = (w_1x_1 + w_2x_2 + \cdots + w_nx_n) + b$$

- Each input x_i has a dedicated **weight (w_i)** that amplifies or suppresses that signal.

- The **bias (b)** acts as an internal baseline, shifting the combined signal up or down.

Step 2: Activation (The Output Filter)

The raw sum z is passed into an **activation function** $f(z)$ to produce the neuron's final output:

$$a = f(z)$$

The activation function decides how strongly this neuron should "fire" its signal forward to the next layer. Without this step, stacking multiple layers would just be adding straight lines together, which collapses mathematically into another straight line.

3. The Biological Parallel: How Real Brain Cells Work

Artificial neural networks borrow their structure directly from biological neurons in the human nervous system:

Biological Element	What It Does in the Body	Artificial Counterpart
Dendrites	Branch-like fibers that receive incoming chemical signals from neighboring cells.	**Inputs (x)**
Synapse Strength	The physical connection gap between neurons; can grow stronger or weaker with experience.	**Weights (w)**
Cell Body (Soma)	Pools and aggregates electrical charges arriving through all dendrites.	**Summation / Aggregation (Σ)**
Axon Hillock	Checks if total incoming charge crosses a physical threshold (about $-55\ \text{mV}$).	**Activation Function (f) & Bias (b)**
Axon	The output cable that fires an electrical spike (*action potential*) to the next cells.	**Neuron Output (a or y)**

A Real-Life Incident: Touching a Warm Mug vs. a Hot Stove

- **Warm Mug:** Your finger's heat receptors send weak signals. Your sensory neurons aggregate these charges, but they barely hover near the threshold. The neurons fire slowly and intermittently, letting you know it is safe to hold.
- **Hot Stove:** Intense thermal energy triggers an huge amount of incoming signals across thousands of dendrites. The total charge at the soma easily blows past the threshold. The neuron fires a rapid, maximum-strength **action potential** down its axon, commanding your hand muscles to pull back instantly.

Biological neurons follow an **"all-or-none" law**: a neuron either fires completely or does not fire at all. Activation functions in artificial networks are a smoothed mathematical version of this firing mechanism.

4. How the Network Learns: Weight Training and Epochs

A neural network does not start out smart; it learns by trial, error, and repetition.

- **Weight Training:**
  1. The network starts with small, randomized weights and biases.
  2. It runs data through the network to make a guess (**Forward Pass**).
  3. It calculates how far the guess is from the correct answer using a loss function.
  4. It works backward to calculate the error share of every weight (**Backpropagation**).
  5. It uses an optimization algorithm (like Gradient Descent) to nudge the weights in the direction that lowers the error.
- **Epoch:** One complete cycle where the **entire training dataset** has passed through the network forward and backward once.
  - *Why we need multiple epochs:* A network cannot absorb every pattern in a single pass. Just as a human needs to read a textbook chapter several times to remember the details, a model needs dozens or hundreds of epochs to gradually steer its weights into their optimal values.

5. Output Activation Functions: Regression vs. Classification

While hidden layers typically use non-linear functions (like ReLU) to capture complex shapes, the **output layer's** activation function is chosen based on the final task:

Linear Activation Function (For Regression)

The linear activation function does not transform or squash the signal at all:

f(z) = z

- **Output Range:** Any real number ($-\infty \text{ to } +\infty$).
- **When to use it:** At the output layer of **Regression** tasks where the target is a continuous, unbounded quantity (e.g., predicting house prices, temperature, revenue, or car speed).

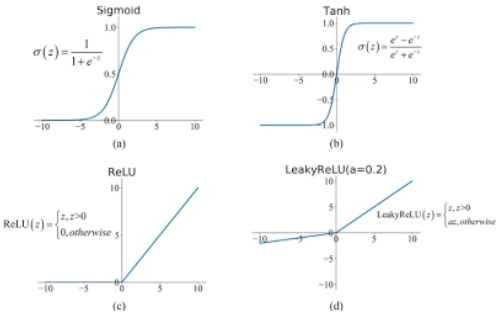
Sigmoid Activation Function (For Binary Classification)

The Sigmoid function compresses the aggregated sum into an S-shaped curve:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Output Range:** Strictly between **0 and 1**.
- **When to use it:** At the output layer of **Binary Classification** tasks. The number represents a probability (e.g., a 0.88 value means an 88% chance an image contains a tumor), allowing a simple cutoff threshold like 0.5 to make the final decision.

Detailed Breakdown of Each Function



1. Linear Activation: $f(z) = z$

The linear activation does not alter the neuron's input; what enters is exactly what leaves.

- **How it behaves:** If the input is 5, the output is 5. If the input is -12.4, the output is -12.4.
- **Best used for:** The **output layer** of regression models, where the goal is to predict an unbounded, continuous value (like house prices or temperatures).
- **Major limitation:** If used in hidden layers, stacking multiple linear layers collapses mathematically into a single linear equation. A network with only linear activations cannot learn curves, boundaries, or complex real-world relationships.

2. Sigmoid Activation: $f(z) = \frac{1}{1+e^{-z}}$

Sigmoid squashes any real number into an S-shaped curve between 0 and 1.

- **How it behaves:** Large positive numbers approach 1, large negative numbers approach 0, and 0 becomes exactly 0.5.
- **Best used for:** The **output layer** of binary classification models, because its output directly maps to a probability (from 0% to 100%).
- **Major limitations:**
  - **Vanishing Gradients:** When inputs are very large or very small, the curve becomes nearly flat. The derivative (slope) in these flat regions drops close to zero, causing backpropagation to stop updating weights in early layers.
  - **Not Zero-Centered:** Because all outputs are positive numbers between 0 and 1, the gradients during backpropagation all take the same sign (all positive or all negative), forcing weights to update in an inefficient zig-zag path.

3. Tanh (Hyperbolic Tangent): $f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

Tanh is a scaled and shifted version of the Sigmoid function that squashes numbers between -1 and $+1$.

- **How it behaves:** It maintains an S-shaped curve like Sigmoid, but crosses $(0, 0)$. Negative inputs yield negative outputs, and positive inputs yield positive outputs.
- **Advantage over Sigmoid:** Because its outputs are centered around zero, the average output is close to 0. This balances weight updates and speeds up training convergence.
- **Major limitation:** It still flattens out completely when inputs are very high or very low, meaning it still suffers heavily from the **vanishing gradient problem** in deep networks.

4. ReLU (Rectified Linear Unit): $f(z) = \max(0, z)$

ReLU acts like a one-way electrical valve: it blocks all negative numbers and allows all positive numbers to pass straight through unchanged.

- **How it behaves:** If $z \leq 0$, the output is 0. If $z > 0$, the output is simply z.
- **Why it dominates modern deep learning:**
  - **Computationally Cheap:** Calculating $\max(0, z)$ requires just a simple `if` check, avoiding expensive exponential calculations (e^z).
  - **No Saturation for Positive Numbers:** For positive values, the slope is a constant 1. Gradients never shrink or vanish, allowing networks with dozens or hundreds of layers to train quickly and reliably.
  - **Sparse Activation:** Turning negative values to zero keeps many neurons dormant at any given time, making the network lightweight and efficient.
- **Major limitation (The "Dying ReLU" Problem):** If a neuron's weights update in a way that its input is always negative, it will consistently output 0 with a slope of 0. That neuron effectively "dies" and permanently stops learning.

Side-by-Side Comparison

Activation Function	Mathematical Formula	Output Range	Zero-Centered?	Vanishing Gradient?
Linear	$f(z) = z$	$(-\infty, +\infty)$	Yes (if input is)	No
Sigmoid	$f(z) = \frac{1}{1+e^{-z}}$	(0, 1)	No (all positive)	Severe at extremes
Tanh	$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	(-1, 1)	Yes	Severe at extremes
ReLU	$f(z) = \max(0, z)$	$[0, +\infty)$	No	None for $z > 0$; freezes if $z \leq$